RAPTOR DEFENSE v2.0 // Spécification de Déploiement Industriel

Architecture de Flotte & Intégration H24 du Gardien Noyau (`rap sentry`) sur Parcs de Production

Dans un parc de serveurs d'entreprise (plusieurs dizaines à plusieurs milliers de nœuds physiques ou virtualisés), la sécurité périmétrique et les agents EDR cloud fournissent la visibilité macroscopique, mais ne garantissent pas l'arrêt instantané d'une attaque sur le stockage local. Si un attaquant obtient une élévation de privilèges ou si un rançongiciel infiltre une machine via une faille *Zero-Day* applicative (sur un serveur Apache, une base MySQL ou un relais de messagerie), le chiffrement s'exécute localement à la vitesse du sous-système d'E/S (disques NVMe/SSD).

Le module `rap sentry` agit comme un disjoncteur différentiel physique décentralisé sur chaque hôte : **il monte la garde 24h/24 et 7j/7 au niveau du noyau Linux (`fanotify`), consomme 0 % de processeur au repos et fige tout processus hostile en moins de 40 microsecondes dès la première tentative d'ouverture d'un appât.**

1. Cartographie des Canaris par Profil de Serveur

L'efficacité du piège repose sur l'emplacement stratégique du fichier leurre `!00_raptor_vault.kdbx`. Parce que son nom débute par un point d'exclamation, les logiciels malveillants itérant par ordre alphabétique l'attaquent en tête de liste. Sur une flotte hétérogène, les leurres doivent être placés au plus près des données critiques de chaque service.

Typologie du Serveur	Emplacements Cibles des Canaris	Justification Métier
Serveurs Web (Apache / Nginx)	`/var/www/`, `/etc/apache2/`, `/etc/nginx/`, `/var/log/`	Protection des racines web, des certificats TLS et des configurations de virtual hosts contre les web-shells.
Bases de Données (MySQL / MariaDB / PostgreSQL)	`/var/lib/mysql/`, `/var/lib/postgresql/`, `/data/db/`	Protection immédiate des volumes de tables brutes et des disques de stockage attachés (SAN/NFS).
Serveurs de Messagerie (Postfix / Dovecot)	`/var/mail/`, `/var/vmail/`, `/home/`	Protection des boîtes de stockage des courriels et des répertoires d'utilisateurs.
Hôtes de Virtualisation / Conteneurs (KVM / Docker)	`/var/lib/docker/volumes/`, `/var/lib/libvirt/images/`	Piège placé sur les points de montage de persistance des conteneurs et images disques qcow2.
Serveurs de Fichiers & Partages (Samba / NFS)	À la racine de chaque montage partagé (`/srv/partage/`)	Première barrière face aux ransomwares ciblant les partages réseaux d'entreprise.

2. Standardisation de l'Exécution Permanente : L'Unité Systemd

En production industrielle, un processus de surveillance ne doit jamais dépendre d'une session de terminal. Il doit être géré par le gestionnaire de services du système d'exploitation (`systemd`) pour garantir son démarrage au boot, sa surveillance de vie et son redémarrage immédiat en cas d'incident matériel.

Créez le fichier de service durci `/etc/systemd/system/raptor-sentry.service` :

```
[Unit]
Description=RAPTOR Defensive Sentry // Surveillance Synchrone Fanotify
H24
Documentation=man:rap-sentry(1)
After=network.target local-fs.target
DefaultDependencies=no
Before=shutdown.target

[Service]
Type=forking
ExecStartPre=/usr/local/bin/rap sentry arm /var/www /var/lib /home /etc
ExecStart=/usr/local/bin/rap sentry run -d --arm
ExecStop=/usr/local/bin/rap sentry disarm
Restart=always
RestartSec=3s

# Privilèges noyau stricts
CapabilityBoundingSet=CAP_SYS_ADMIN CAP_SYS_PTRACE CAP_KILL
CAP_NET_ADMIN
AmbientCapabilities=CAP_SYS_ADMIN CAP_SYS_PTRACE CAP_KILL CAP_NET_ADMIN

# Immunité totale contre l'épuisement mémoire (OOM Killer)
OOMScoreAdjust=-1000

# Verrouillage de la mémoire physique (évite le swap)
LimitMEMLOCK=infinity

# Confinement du service
ProtectSystem=full
ProtectHome=false
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

3. Automatisation du Déploiement à l'Échelle (Ansible Playbook)

Pour orchestrer le déploiement de RAPTOR sur 500 serveurs simultanément sans intervention manuelle, voici le playbook Ansible standardisé `deploy_raptor_fleet.yml` :

```yaml
---
- name: Déploiement Industriel de RAPTOR Sentry H24
  hosts: all_production_servers
  become: yes
  vars:
    raptor_archive: "raptor-v2.0.0-Linux-x86_64.tar.gz"
    install_prefix: "/usr/local"

  tasks:
    - name: 1. Déposer l'archive binaire compilée sur le nœud cible
      ansible.builtin.copy:
        src: "{{ raptor_archive }}"
        dest: "/tmp/{{ raptor_archive }}"
        mode: '0600'

    - name: 2. Extraire les binaires et bibliothèques
      ansible.builtin.unarchive:
        src: "/tmp/{{ raptor_archive }}"
        dest: "/tmp/"
        remote_src: yes

    - name: 3. Exécuter l'installation standardisée des binaires et
capacités
      ansible.builtin.command:
        cmd: "./install.sh --quiet"
        chdir: "/tmp/raptor-v2.0.0-Linux-x86_64"
      register: install_result
      changed_when: "'Installation achevee' in install_result.stdout"

    - name: 4. Nettoyer les fichiers temporaires
      ansible.builtin.file:
        path: "{{ item }}"
        state: absent
      loop:
        - "/tmp/{{ raptor_archive }}"
        - "/tmp/raptor-v2.0.0-Linux-x86_64"

    - name: 5. Déployer l'unité de service systemd durcie
      ansible.builtin.template:
```

```
        src: "templates/raptor-sentry.service.j2"
        dest: "/etc/systemd/system/raptor-sentry.service"
        owner: root
        group: root
        mode: '0644'

    - name: 6. Recharger systemd et activer RAPTOR Sentry H24 au
démarrage
      ansible.builtin.systemd:
        name: raptor-sentry
        daemon_reload: yes
        state: started
        enabled: yes

    - name: 7. Valider l'intégrité opérationnelle du dispositif
      ansible.builtin.command:
        cmd: "/usr/local/bin/rap sentry status"
      register: sentry_status
      failed_when: "'ACTIF' not in sentry_status.stdout"
      changed_when: false
```

4. Supervision de Flotte, Métriques & Remontée SIEM

Dans une architecture d'entreprise, les équipes SOC doivent être informées de l'état du parc sans charger les serveurs.

A. Télémétrie d'état passive (Zabbix / Prometheus / Datadog)

Pour vérifier que la sentinelle est bien en veille sur chaque nœud, configurez une sonde locale exécutant :

/usr/local/bin/rap sentry status

- **Indicateur sain (Code sortie POSIX 0) :** Tous les canaris sont intègres (taille stricte de 32 768 octets), le watcher `fanotify` est actif.
- **Alerte (Code sortie > 0) :** Canari manquant ou service non chargé.

B. Traitement d'une détonation (L'alarme critique)

Lorsqu'un ransomware tente de chiffrer un canari sur un serveur Apache ou une base MySQL :

1. Le noyau Linux transmet l'interception à RAPTOR.
2. RAPTOR bloque l'accès (`FAN_DENY`), paralyse l'attaquant (`SIGSTOP`), coupe ses descripteurs réseau et l'abat (`SIGKILL`) en moins de 40 microsecondes.

3. Les données médico-légales de l'attaquant (PID, PPID, chemin de l'exécutable, utilisateur, latence d'interception) sont inscrites instantanément dans l'anneau de mémoire vive autonome `/dev/shm/raptor_blackbox`.
4. Le service journalise l'événement via `syslog` / `journald`, qui l'expédie immédiatement vers votre SIEM central (Splunk, Elastic, Sentin

CRITICAL [RAPTOR_SENTRY] Threat intercepted and eliminated on /var/www/!00_raptor_vault.kdbx | PID: 18452 (cmd: ./.encrypt) | Signal: SIGKILL | Latency: 38us | Action: NET_REVOKE + PURGE

5. Procédures de Maintenance et Fenêtres de Sauvegarde

Les opérations régulières d'administration (sauvegardes massives de bases de données, archivage `tar`, indexation antivirus générale) peuvent inspecter les répertoires contenant les leurres sans intention malveillante.

Pour éviter toute interruption accidentelle durant ces opérations planifiées, la procédure opérationnelle standard (SOP) consiste à encadrer les scripts de maintenance par le cycle de désarmement :

```
# 1. Avant le début de la sauvegarde de masse :
systemctl stop raptor-sentry

# 2. Exécution du backup (les canaris sont supprimés, aucun risque de déclenchement)
rsync -aAXv /var/www/ /mnt/backup/www/

# 3. Réactivation immédiate après la sauvegarde :
systemctl start raptor-sentry
```

Le service recrée automatiquement tous les fichiers coffre-forts intègres, réinitialise les verrous `fanotify` et remet le serveur sous cloche protectrice en une fraction de seconde.

Ce modèle d'architecture permet de déployer RAPTOR sur un parc hétérogène en tant que rempart silencieux et permanent : les services applicatifs (Apache, bases de données, messageries) continuent de tourner à pleine charge sans aucune dégradation de performance, pendant que le noyau Linux veille au grain pour désamorcer toute tentative de sabotage à la source.